

Exer 1)

```
=== Q1: BLOCOS (Sequencial np=1) ===
Processus 0 : a calculé les lignes 0 à 1024 en 0.6670s

--- Temps Total MPI (1 processus) : 0.6850s ---
Image sauvegardée sous 'mandelbrot.png'

=== Q1: BLOCOS (Paralelo np=4) ===
Processus 0 : a calculé les lignes 0 à 256 en 0.1729s
Processus 3 : a calculé les lignes 768 à 1024 en 0.1740s
Processus 2 : a calculé les lignes 512 à 768 en 0.2048s
Processus 1 : a calculé les lignes 256 à 512 en 0.2135s

--- Temps Total MPI (4 processus) : 0.2295s ---
Image sauvegardée sous 'mandelbrot.png'

=== Q2: CÍCLICO (Sequencial np=1) ===
Temps total de calcul (MPI Cyclique) : 0.6656s
Image sauvegardée sous 'mandelbrot_cyclic.png'

=== Q2: CÍCLICO (Paralelo np=4) ===
Temps total de calcul (MPI Cyclique) : 0.2023s
Image sauvegardée sous 'mandelbrot_cyclic.png'

=== Q3: DINÂMICO (Simulação Seq: 1 Mestre + 1 Escravo) ===
Démarrage Maître-Esclave avec 1 travailleurs.
Temps total (Maître-Esclave Dynamique) : 0.7614s
Image sauvegardée sous 'mandelbrot_dynamic.png'

=== Q3: DINÂMICO (Paralelo: 1 Mestre + 4 Escravos) ===
Démarrage Maître-Esclave avec 4 travailleurs.
Temps total (Maître-Esclave Dynamique) : 0.2143s
Image sauvegardée sous 'mandelbrot_dynamic.png'
```

Stratégie	Temps Séquentiel (Tseq)	Temps Parallèle (Tpar)	Nb. de Travailleurs	Accélération (Speedup)
Q1. Par Bloc (Blocs équitables)	0.6850s	0.2295s	4	2.98
Q2. Cyclique (Répartition statique)	0.6656s	0.2023s	4	3.29
Q3. Dynamique (Maître-Esclave)	0.7614s*	0.2143s	4**	3.55

R1.1)

L'accélération obtenue (**2.98**) est inférieure à l'idéal théorique de 4. Cela s'explique par un fort déséquilibre de charge inhérent à la répartition par blocs.

Comme le montre le détail des temps d'exécution, les processus traitant le centre de l'image (Rangs 1 et 2) mettent environ **20% de temps en plus** que ceux traitant les bords (Rangs 0 et 3). Les processus du centre doivent calculer la zone complexe de la fractale (nombreuses itérations), tandis que les processus des bords traitent majoritairement du vide (divergence rapide). Le temps global du programme est donc limité par le processus le plus lent.

R1.2)

Perte de localité spatiale (Cache Efficiency) : Dans la répartition par blocs, un processeur travaille sur une zone mémoire contiguë, ce qui optimise l'utilisation du cache CPU. Avec la répartition cyclique, l'accès mémoire est fragmenté. Cela peut augmenter le nombre de **défauts de cache** et réduire l'efficacité des calculs vectoriels.

R1.3)

1. **Comparaison des performances:** L'accélération obtenue (**3.55**) est nettement supérieure à celle de la partition par blocs (**2.98**), et dépasse même légèrement la stratégie cyclique (**3.29**), démontrant une excellente scalabilité.

2. **Analyse:** La stratégie dynamique offre un équilibrage de charge quasi parfait. Cependant, si l'on regarde le **temps absolu**, la stratégie cyclique reste la plus rapide (0.20s contre 0.21s). Cela s'explique par le **surcoût de communication** (overhead) du modèle Maître-Esclave. Le dialogue constant pour distribuer les tâches génère une latence que la stratégie cyclique (répartition statique immédiate) n'a pas."

Exer 2)

```
=== MATVEC : COLONNES (Séquentiel np=1) ===  
COLONNE - Processus : 1 | Temps : 0.0092s  
v[0] = 22914881536.0  
  
=== MATVEC : COLONNES (Parallèle np=4) ===  
COLONNE - Processus : 4 | Temps : 0.0036s  
v[0] = 22914881536.0  
  
=== MATVEC : LIGNES (Séquentiel np=1) ===  
LIGNE - Processus : 1 | Temps : 0.0096s  
v[0] = 22914881536.0  
  
=== MATVEC : LIGNES (Parallèle np=4) ===  
LIGNE - Processus : 4 | Temps : 0.0035s  
v[0] = 22914881536.0
```

1. Validation Numérique

Les résultats obtenus sont cohérents. La valeur de contrôle $v[0]$ (2.29×10^{10}) est identique pour les exécutions séquentielles et parallèles dans les deux stratégies, ce qui valide l'exactitude des algorithmes parallèles.

2. Performance et Speedup (pour N=4096)

- **Décomposition par Colonnes :**
 - Temps Séquentiel : 0.0092 s
 - Temps Parallèle (4 proc) : 0.0036 s

- **Speedup : 2.55**
- **Décomposition par Lignes :**
 - Temps Séquentiel : 0.0096 s
 - Temps Parallèle (4 proc) : 0.0035 s
 - **Speedup : 2.74**

Conclusion :

Les deux approches offrent une bonne accélération. La stratégie par **lignes** s'est montrée légèrement plus performante (2.74 vs 2.55). Cela s'explique par le fait que la communication collective **Allgather** (utilisée dans l'approche par lignes) est souvent moins coûteuse en temps CPU que l'**Allreduce** (utilisée dans l'approche par colonnes), car elle ne nécessite pas d'opérations arithmétiques pendant le transfert de données.

Exer 3)

R3.1 : Prédiction de l'accélération maximale (Loi d'Amdahl)

D'après la loi d'Amdahl, l'accélération maximale théorique est limitée par la portion séquentielle du programme qui ne peut pas être parallélisée. Ici, la partie parallèle représente 90% (0.9) du temps. La partie séquentielle (s) est donc de 10% (0.1).

L'accélération maximale, même avec un nombre infini de processeurs, se calcule par la formule : $S_{\text{max}} = 1 / s$ $S_{\text{max}} = 1 / 0.1 = 10$

Alice peut donc espérer une accélération maximale théorique de 10.

R3.2 : Nombre raisonnable de nœuds de calcul

Il faut trouver un compromis entre le gain de vitesse et l'efficacité (pour ne pas gaspiller de ressources CPU).

Si l'on utilise 9 nœuds : L'accélération est d'environ 5. L'efficacité est donc de 55% (5 pour 9 nœuds). C'est le point où le temps passé dans la partie parallèle devient égal au temps de la partie séquentielle.

Si l'on passe à 19 nœuds : L'accélération monte seulement à environ 6.8, mais on utilise deux fois plus de machines, ce qui gaspille beaucoup de ressources.

Il semble donc raisonnable de choisir entre 8 et 10 nœuds. Au-delà de 9 nœuds, le gain de performance devient marginal par rapport au coût du matériel ajouté.

R3.3: Accélération avec la loi de Gustafson (Doublement des données)

La loi de Gustafson suppose que si l'on augmente la puissance de calcul, on augmente aussi la taille du problème traité. Ici, on double la quantité de données. Supposons que le temps séquentiel initial soit de 1 unité et le temps parallèle de 9 unités (total 10 unités, soit 90% parallèle).

Si on double les données, la partie séquentielle reste fixe (1 unité), mais la partie parallèle double (elle passe de 9 à 18 unités). Le nouveau temps total est de 19 unités (1 + 18). La nouvelle portion séquentielle devient $1 / 19$ (environ 5.2%). La nouvelle accélération maximale espérée devient : $S_{\text{max}} = 1 / (1/19) = 19$. En doublant la taille du problème, Alice peut espérer une accélération maximale théorique de